03_BUILD_FROM_SCRATCH_WALKTHROUGH

Pintop | 2026 Training

Build something from a blank file - Rate Limiting & Performance Measurement

Pintop Technologies Limited - runnable or integration-based micro-lab

This exercise complements the real codebase reference in `01_CODE_ALONG_LAB.md`. The previous exercise rebuilds and explains existing course code; this one creates something **new**. For beginners, the aim is to see every character typed, then see a result.

Create the file and type each line

1. Create a **disposable copy of the course project**; open the real Laravel `routes/api.php` in that copy.
2. Read the following code with your students. Explain what the HTTP method, URL, closure inputs and returned value mean.
3. Type the `Route` declaration **at the bottom of** `routes/api.php` (do not paste the opening `<?php` a second time). Save and run `php -l routes/api.php` from the Laravel application directory.
4. Start `php artisan serve --port=8000` in a separate terminal. Run the requests below **from a second terminal**. Always check `php artisan route:list --path=api` if the URL is not found.
5. Change one test input, predict the result, re-run, and restore the clean copy after the lesson.

Ready-to-type example (`03_CREATE_A_WORKING_LARAVEL_ROUTE.php`):

```
\Illuminate\Support\Facades\Route::get('/classroom/10-rate-limiting-performance-', function () {
    return response()->json(['lesson' => 'Rate Limiting & Performance Measurement', 'status' => 'ok']);
})->middleware('throttle:3,1');
```

Run and inspect the result

```
for i in 1 2 3 4; do curl -s -o /dev/null -w '%{http_code}\n' 'http://127.0.0.1:8000/api/classroom/10-rate-
```

Expected result: On the same local client, repeated requests should eventually receive HTTP 429 if the middleware is active.

Say this to learners

"When we build software, we don't start by typing at random. We start with a result we can describe. We then write the smallest code that can give us that result. I will type the code slowly and explain every name. When it runs, you will change one thing and use the output to prove what you understood."

Mini checkpoint

1. Explain one line that changes the output.
2. Predict a changed result before editing.
3. Run once with a normal input and once with an invalid, empty or changed input.
4. Put screenshots or the terminal output into your class evidence notes.

Note: The demo is intentionally small and designed for local teaching. Follow the class's separate real-codebase exercise for realistic application structure and security.